MLDS Assignment 1

From initial policy-gradient failures to controlled improvements
Complete report | 23 October 2026

GitHub username: Dorrie-ge
Repository: mlds-assignment1-rl
https://github.com/Dorrie-ge/mlds-assignment1-rl

Main findings

My main lesson from this assignment is that a program can execute correctly without learning a useful policy. The shuttle solution was consistent across methods and gave a dispatch threshold of 13 passengers. The games were less straightforward. A smaller learning rate helped CartPole, but Pong stayed close to -21 even after I increased the training budget. That led me to examine the update weighting, image representation and actor learning rate instead of assuming that more episodes would be enough.

Core experiment (strict discounted objective)	Train	500-episode mean +/- SD
CartPole-v1, b=none	2000	300.240 +/- 97.968
CartPole-v1, b=value	2000	220.218 +/- 45.618
ALE/Pong-v5, b=none	1000	-20.390 +/- 0.827
ALE/Pong-v5, b=value	1000	-20.720 +/- 0.564

Improved Pong extension: lr=0.001 reaches -7.742 +/- 6.311 on selection seed 0. The two independent confirmation seeds average -9.641. Across the three policy means, the average is -9.008 and the sample SD is 1.164. Scores remain negative on average; this is improved learning, not stable mastery.

Objective distinction. Core experiments use the exact start-state discounted objective requested in the handout. Later Pong extensions retain gamma=0.99 inside reward-to-go but remove the outer gamma^t actor weight. This is a different update weighting and is reported explicitly as a supplementary method, not silently substituted for the core objective.

Reading order: sections 1-3 define the model and algorithms; sections 4-7 contain all four required plot pairs; sections 8-13 reconstruct the improvement experiments; sections 14-18 cover substantive validation, edge cases, execution instructions and future work. Appendix A inventories every completed experiment.

1. Shuttle MDP and solution methods

The decision state s is the number of passengers waiting before dispatch: s in {0,...,200}. The action a is 0 (wait) or 1 (dispatch one shuttle). Capacity K=15, dispatch cost c_f=100, holding cost c_h=2, discount gamma=0.95 and independent arrivals A uniformly distributed on {1,2,3,4,5} follow the assignment.

Timing and boundary assumptions

```
left(s,a) = max(s - 15*a, 0)
c(s,a) = 100*a + 2*left(s,a)
s_next = min(200, left(s,a) + A)
```

I modeled dispatch before holding cost and arrivals, because the handout charges holding cost to passengers left after dispatch. I cap the next queue at 200 and discard overflow without an extra charge. Empty dispatch is allowed but still costs 100, and ties choose waiting. These are explicit modeling choices: a different overflow penalty or event order would define a different problem. With independent arrivals, the current queue contains the information needed for the next decision.

Finite horizon: enumeration with T=500

```
J_500(s) = 0
J_t(s) = min_a {c(s,a) + 0.95*(1/5)*sum_A J_(t+1)(s_next)}
```

For enumeration, I worked backward from a zero terminal cost at T=500, considering all 201 states, both actions and all five arrival outcomes at each step. I interpret enumeration as this finite-horizon Bellman calculation. Enumerating 2^500 open-loop action strings would be impractical and would not describe every state-dependent policy. To check this interpretation computationally, I also compared the calculation with a direct recursive tree on a small problem, as reported in section 14.

Infinite horizon: VI and PI

Value iteration starts from zero and applies the discounted Bellman minimum operator until the maximum update residual is below 1e-10. Policy iteration exactly evaluates the current policy with (I-0.95 P_pi)J=c_pi, then greedily improves it until no action changes. Both methods minimize positive costs; a reward formulation would use r=-c and V=-J.

Outcome	Measured value
VI iterations	518
PI iterations	6
Optimal cost J*(0)	576.670167764
Infinite-horizon policy	Wait at 0-12; dispatch at 13-200

2. Required shuttle plots

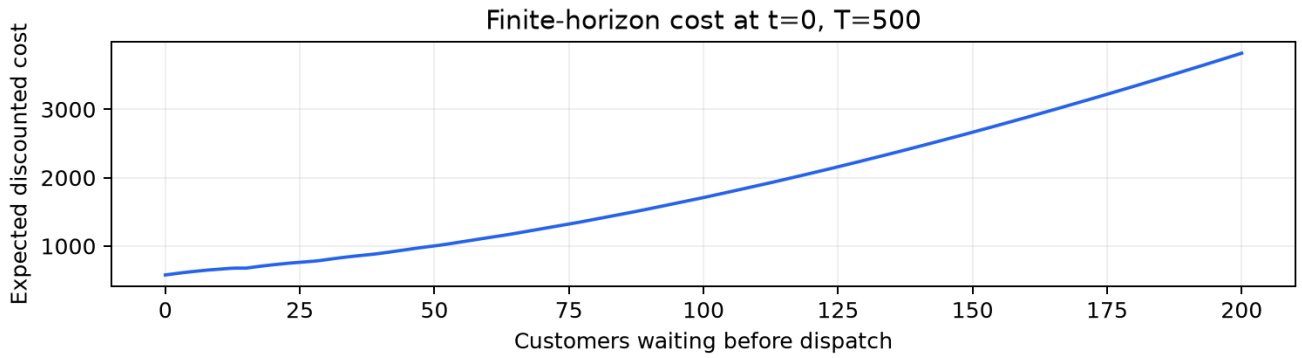


Figure 1. Enumeration / backward induction: optimal positive discounted cost at time 0, horizon T=500.

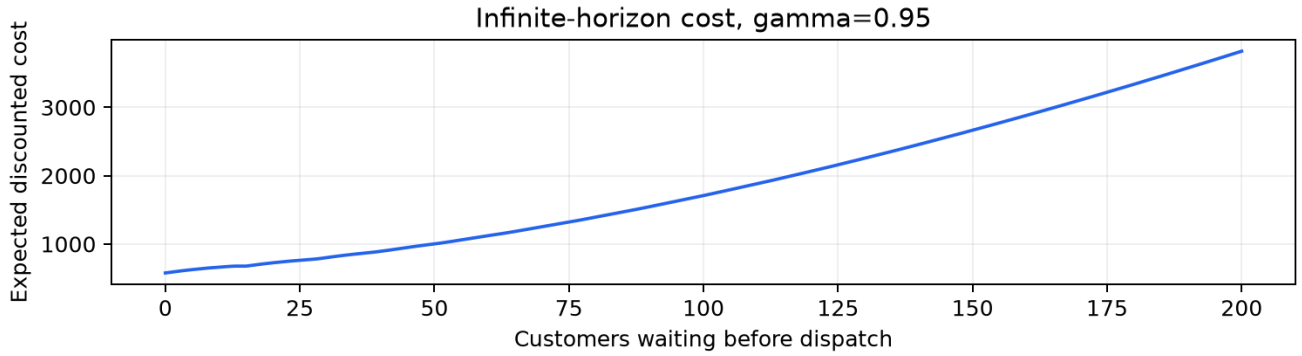


Figure 2. Value iteration: optimal infinite-horizon positive discounted cost.

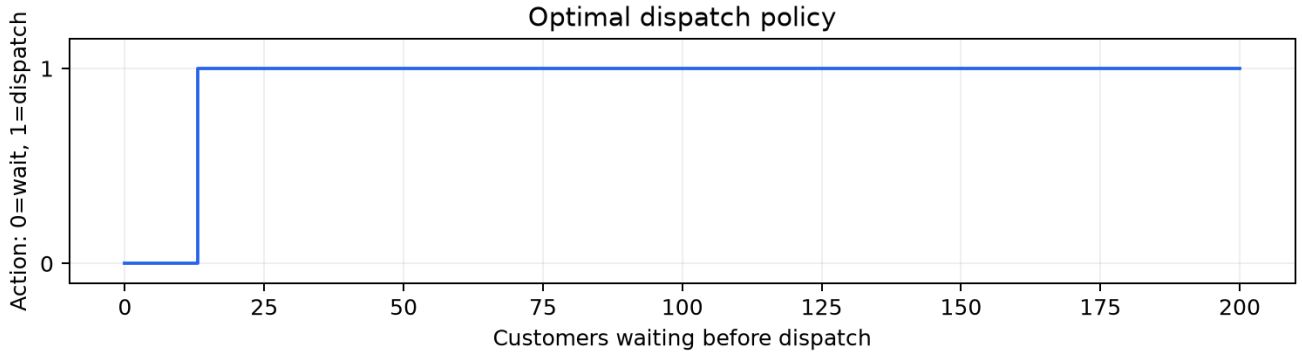


Figure 3. Policy iteration: action 0=wait, action 1=dispatch; threshold 13. All queue states 0-200 are shown.

I initially found the threshold of 13 interesting because the shuttle holds 15 people. Waiting for a full shuttle is not always cheapest: the extra holding cost can outweigh the benefit of serving two more passengers. The three plots are consistent with that tradeoff. T=500 values are within 3e-9 of the infinite-horizon values, although decisions near the finite terminal time can still differ. I retained the numerical values in results/shuttle/values.csv so this agreement can be checked directly.

3. Policy gradient and the value baseline

Core objective and estimator

```
J(theta) = E[sum_t gamma^t * r_t]
G_t = sum_(k=t)^(T-1) gamma^(k-t) * r_k
L_actor = -(1/B) * sum_episodes sum_t gamma^t * log pi_theta(a_t|s_t) * (G_t - b(s_t))
```

I used a categorical policy so that the network learns action probabilities rather than a fixed action rule. PyTorch computes the actor gradient and Adam updates the weights. I kept full-episode reward-to-go, including rewards across Pong score points. CartPole uses gamma=0.95 and Pong gamma=0.99. One distinction became important during debugging: the training objective discounts rewards, while the evaluation score is the undiscounted whole-game sum. Improving one does not automatically imply improving the other.

Part 2: nonconstant, learned state-value baseline

For Part 2, I chose a learned state-value baseline because different game states can have different expected future returns. A separate network b_phi(s) predicts Monte Carlo reward-to-go. I compute its prediction before fitting the current trajectory or batch and detach it from the actor graph. The critic has its own Adam optimizer and squared-error loss. CartPole fits it on trajectory chunks; batched Pong takes one averaged critic update after the actor update, with each trajectory normalized by its length. This keeps the baseline from changing merely to lower the actor loss.

```
E_a[b(s) * grad log pi(a|s)] = b(s) * grad sum_a pi(a|s) = 0
```

The equation explains what I should expect from a baseline: it cancels in the expected score gradient because it does not depend on the sampled action. It may reduce sampling variance, but it does not promise a better game score. I therefore checked both claims separately: an exactly enumerable problem for the expected gradient and frozen-policy trajectories for the variance (section 15b).

Setting	CartPole core	Pong core
Observation / actor	4 coordinates; 128 tanh units; 2 logits	2 binary 80x80 frames; CNN; 2 logits
CNN (Pong only)	Not applicable	Conv 16: 8x8/stride4; Conv 32: 4x4/stride2; FC128/ReLU
Actor / critic lr	0.001 / 0.001	0.0001 / 0.0003
Episodes / batch	2,000 / 1	1,000 / 8
Optimizer / hardware	Adam; CPU, 1 thread	Adam; Apple MPS, 1 CPU thread

Environment IDs are CartPole-v1 and ALE/Pong-v5, the Gymnasium/ALE registration of the requested Pong-v5. Pong retains the default frameskip=4 and sticky-action probability=0.25. The policy maps its two logits only to actions 2 and 3. It never chooses FIRE or NOOP. Episode history resets on termination or truncation. [R2-R4]

4. Part 1: CartPole without baseline

Configuration: CartPole-v1, baseline=none, seed 0, gamma=0.95, actor Adam lr=0.001. Training: 2,000 episodes. Frozen stochastic-policy evaluation: 500 episodes.

Mean score	Sample SD	Random-policy mean
300.240	97.968	21.906



Figure 4. Training reward and trailing 100-episode mean. Source: results/cartpole_none_lr0001/training.csv.

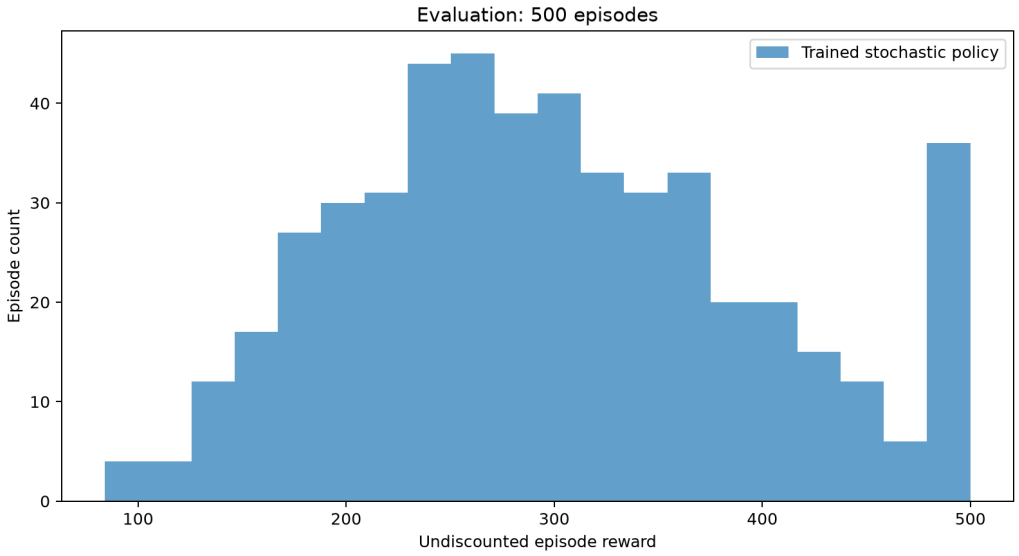


Figure 5. Histogram of the 500 frozen-policy game scores; random-policy reference is displayed separately. Standard deviations use ddof=1.

With lr=0.01, the no-baseline CartPole policy performed worse than random. Reducing lr to 0.001 raised this seed-0 evaluation mean from 9.370 to 300.240. I take that as evidence that the original step size was unsuitable for this run. The broad histogram also matters: the learned policy is useful, but it does not consistently stay balanced for all 500 steps. I kept stochastic sampling during evaluation.

5. Part 2: CartPole with value baseline

Configuration: CartPole-v1, baseline=value, seed 0, gamma=0.95, actor Adam lr=0.001. Training: 2,000 episodes. Frozen stochastic-policy evaluation: 500 episodes.

Mean score	Sample SD	Random-policy mean
220.218	45.618	21.906



Figure 6. Training reward and trailing 100-episode mean. Source: results/cartpole_value_lr0001/training.csv.

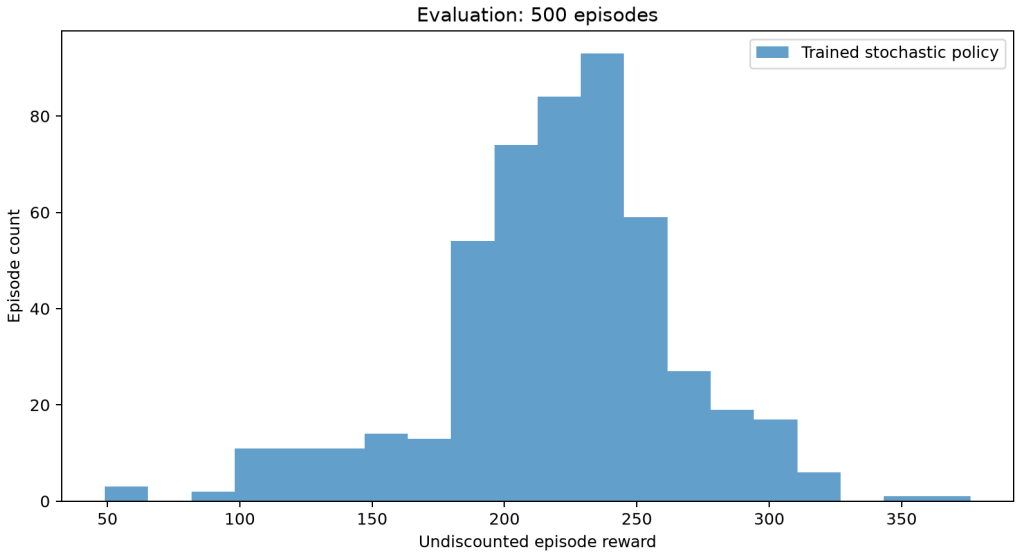


Figure 7. Histogram of the 500 frozen-policy game scores; random-policy reference is displayed separately. Standard deviations use ddof=1.

I expected the baseline to help, so the seed-0 score of 220.218 was a useful caution: it is below the matched no-baseline result. Across three seeds the baseline has the higher average, but neither comparison says it will win on every seed. This is why I measured gradient variance directly rather than using a better-looking reward curve as proof that the baseline works.

6. Part 1: Pong without baseline

Configuration: ALE/Pong-v5, baseline=none, seed 0, gamma=0.99, actor Adam lr=0.0001. Training: 1,000 episodes. Frozen stochastic-policy evaluation: 500 episodes.

Mean score	Sample SD	Random-policy mean
-20.390	0.827	-20.514

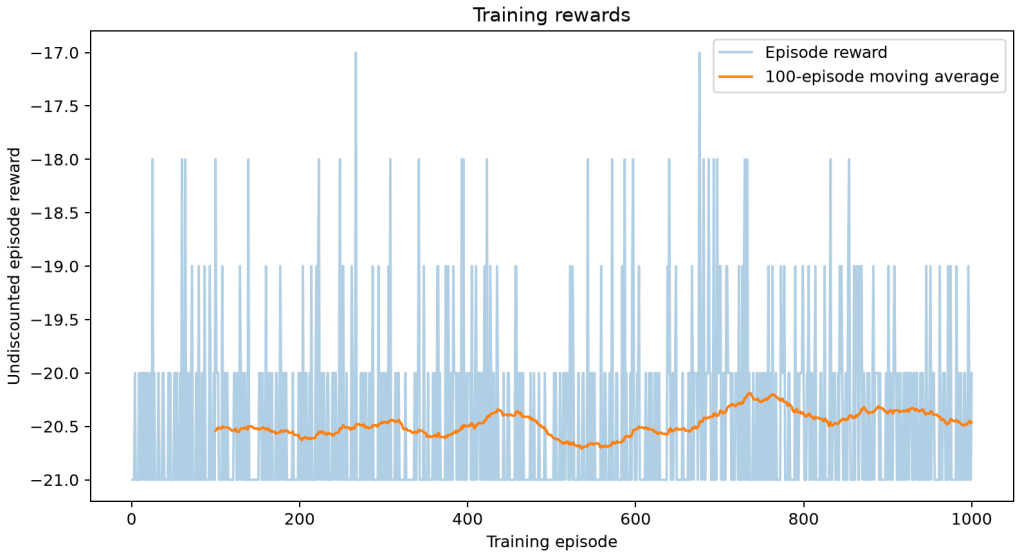


Figure 8. Training reward and trailing 100-episode mean. Source: results/pong_batch_none/training.csv.

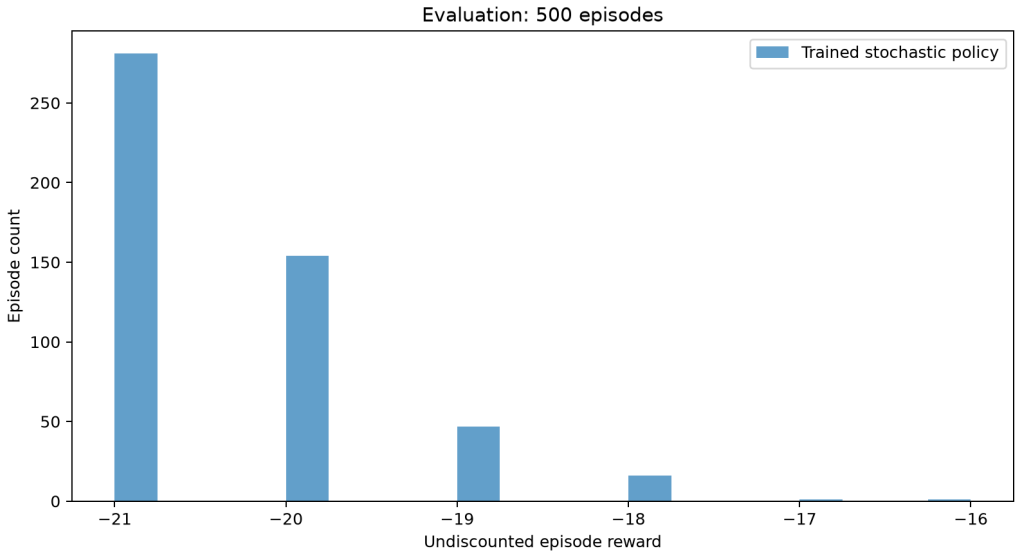


Figure 9. Histogram of the 500 frozen-policy game scores; random-policy reference is displayed separately. Standard deviations use ddof=1.

For this retry, I averaged eight complete Pong episodes per update, initialized the actor output close to an even action distribution and clipped the gradient norm at 5. The result is still near -21. I kept the failed curve and histogram because they show that these stability changes were not enough to learn effective play. Later sections explain the additional experiments that this failure motivated.

7. Part 2: Pong with value baseline

Configuration: ALE/Pong-v5, baseline=value, seed 0, gamma=0.99, actor Adam lr=0.0001. Training: 1,000 episodes. Frozen stochastic-policy evaluation: 500 episodes.

Mean score	Sample SD	Random-policy mean
-20.720	0.564	-20.514

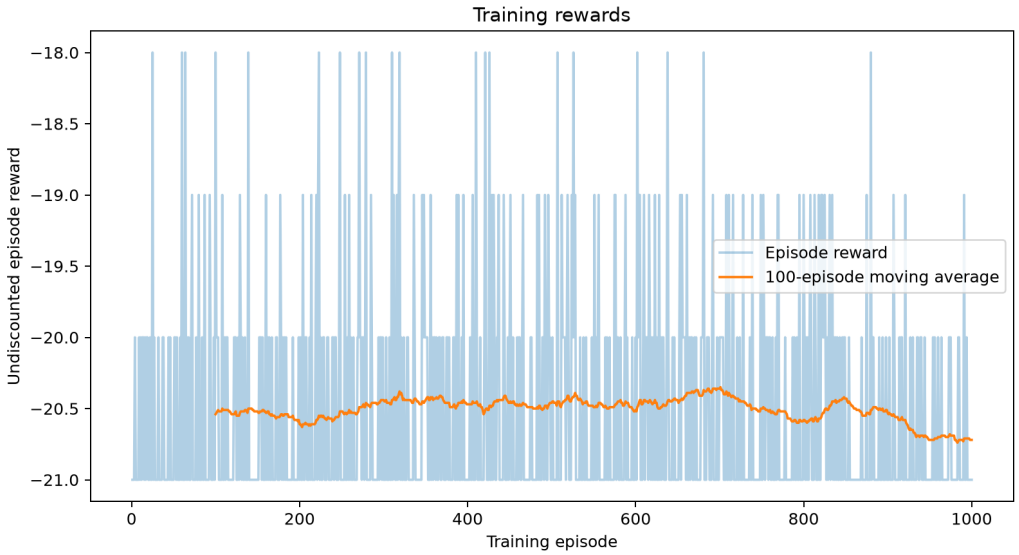


Figure 10. Training reward and trailing 100-episode mean. Source: results/pong_batch_value/training.csv.

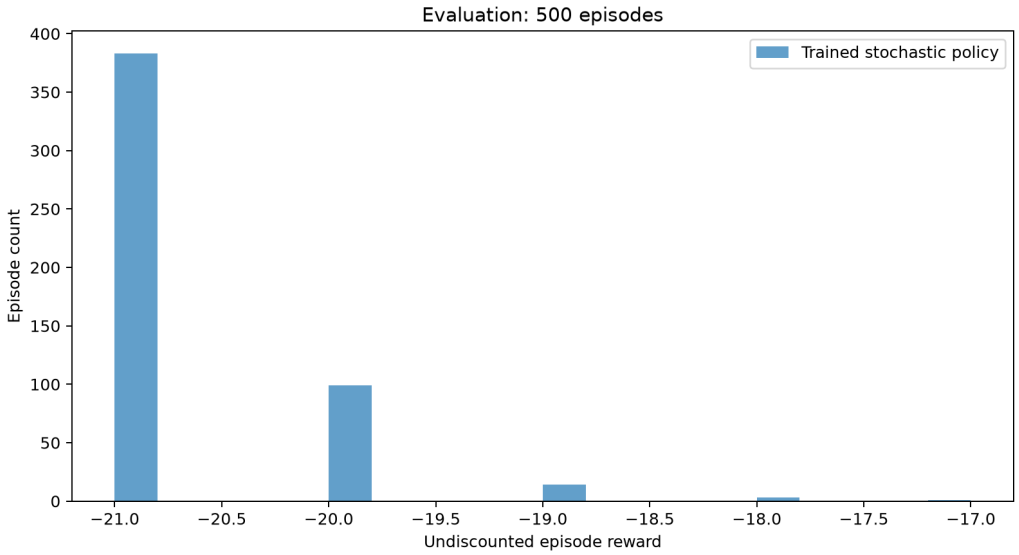


Figure 11. Histogram of the 500 frozen-policy game scores; random-policy reference is displayed separately. Standard deviations use ddof=1.

Adding the CNN value baseline did not fix this Pong failure: its score is slightly below the matched no-baseline score. The critic is detached during the actor update and fitted afterward. My later tests do find lower gradient variance at this checkpoint. I learned to separate those two observations: a less noisy gradient can still make poor progress on the game.

8. Starting point and CartPole improvements

I started with the handout actor learning rate, 0.01, and basic single-episode REINFORCE with the exact outer discount factor. The first full runs used 2,000 training episodes for each CartPole method and 200 for each Pong method, followed by 500 evaluation episodes per policy. Small smoke tests were kept separate. At this point I wanted to see whether the basic implementation learned before adding more changes.

Initial experiment	Actor lr	Train	500-episode mean +/- SD
cartpole_none	0.01	2000	9.370 +/- 0.755
cartpole_value	0.01	2000	170.168 +/- 11.184
pong_none	0.01	200	-21.000 +/- 0.000
pong_value	0.01	200	-21.000 +/- 0.000

The initial results changed that plan. CartPole without a baseline fell below its random reference, while both Pong policies scored -21 with no evaluation variation. I explored lr=0.003 for plain CartPole and obtained 171.432, then compared lr=0.001 with and without the baseline at seeds 0, 1 and 2. The task, architecture and objective stayed the same, which makes this a more informative comparison than changing several settings at once.

Training seed	No baseline mean +/- SD	Value baseline mean +/- SD
0	300.240 +/- 97.968	220.218 +/- 45.618
1	92.460 +/- 31.626	362.986 +/- 121.367
2	106.554 +/- 28.962	214.266 +/- 69.583

none: mean of the three policy means = 166.418; between-training-seed sample SD = 116.107.

value: mean of the three policy means = 265.823; between-training-seed sample SD = 84.198.

These runs gave me a reason to use the smaller CartPole learning rate in the main results. They also showed how much the conclusion depends on the training seed. I would not argue that the baseline is always better from one favorable run. The evaluation scores used during this exploration helped choose the settings, so they are not an untouched final test set.

CartPole task-level sanity check

Deterministic diagnostic (20 episodes each)	Mean reward	Episode ending
Always push left	9.15	Physical termination
Angle + 0.15*angular-velocity feedback	500.00	All 20 reach the 500-step time limit

The predictions are rapid failure under a fixed push and sustained balance under state feedback. Both are observed on separate seeds starting at 9,500,000; reward equals episode length. First-10-step traces retain states, actions and ending flags. This validates state/action conventions, the reward scale and time-limit handling. The controller is hand-coded and is not substituted for a learned-policy score.

9. Pong: stability retry and larger budgets

My first response to Pong was to make training more stable: I lowered actor lr to 0.0001, averaged eight full trajectories, initialized near equal action probabilities and clipped gradients. I retained the CNN, stacked frames and exact gamma^t weighting. Since several stabilizers changed together, I cannot identify which one caused the small change from the original failure.

I then tested the idea that Pong mainly needed more training. I copied the 1,000-episode actor, critic, Adam state and CPU sampling RNG into separate directories and resumed to 3,000 and then 5,000 episodes. The original CSVs remain exact prefixes, and each endpoint has 500 evaluation episodes. This isolates budget within this continuation protocol instead of comparing unrelated fresh runs.

Total episodes	Actor updates	No baseline mean +/- SD	Value baseline mean +/- SD
1000	125	-20.390 +/- 0.827	-20.720 +/- 0.564
3000	375	-20.952 +/- 0.214	-20.548 +/- 0.693
5000	625	-20.724 +/- 0.563	-20.170 +/- 0.916

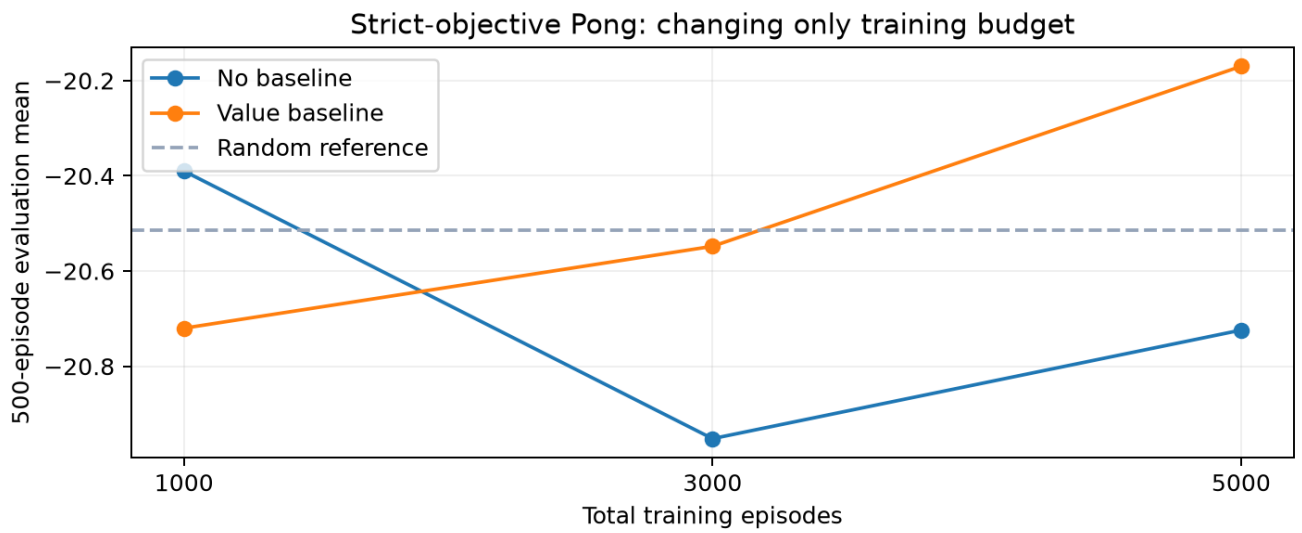


Figure 12. Measured evaluation means at three budgets; each point represents a separately evaluated endpoint, not an inferred score.

At 3,000 episodes the no-baseline result deteriorated; the baseline improved only slightly. At 5,000 both remain poor. Thus this configuration did not become successful merely by extending training. The experiment does not prove that more data can never help; it demonstrates that budget alone was not an effective fix over the tested range. Subsequent experiments hold each fresh-run budget at 3,000 episodes.

10. Pong: the actor time-weighting experiment

Long Pong episodes made me look more closely at the objective. With gamma=0.99, the outer weight is only about 0.00657 at step 500 and 0.0000432 at step 1,000. That is correct for the stated discounted objective, but it gives very little weight to late-game actions. I tried removing this outer factor as an explicitly separate experiment; I did not treat it as a correction to a coding mistake.

```
Core: loss = -sum_t gamma^t * log pi(a_t|s_t) * (G_t - b(s_t))
Extension: loss = -sum_t log pi(a_t|s_t) * (G_t - b(s_t))
```

The extension keeps full-episode gamma=0.99 reward-to-go and removes only the outer time factor. Both branches start from the same 3,000-episode checkpoint and train to 5,000; uniform weighting starts at episode 3,001. Architecture, learning rates, batch, clipping and seed remain fixed. This is a warm-start experiment with inherited optimizer state.

Baseline	Original weight mean	Uniform weight mean	Gain (approx. paired 95% CI)
none	-20.724	-20.444	0.280 [0.194, 0.366]
value	-20.170	-19.954	0.216 [0.090, 0.342]

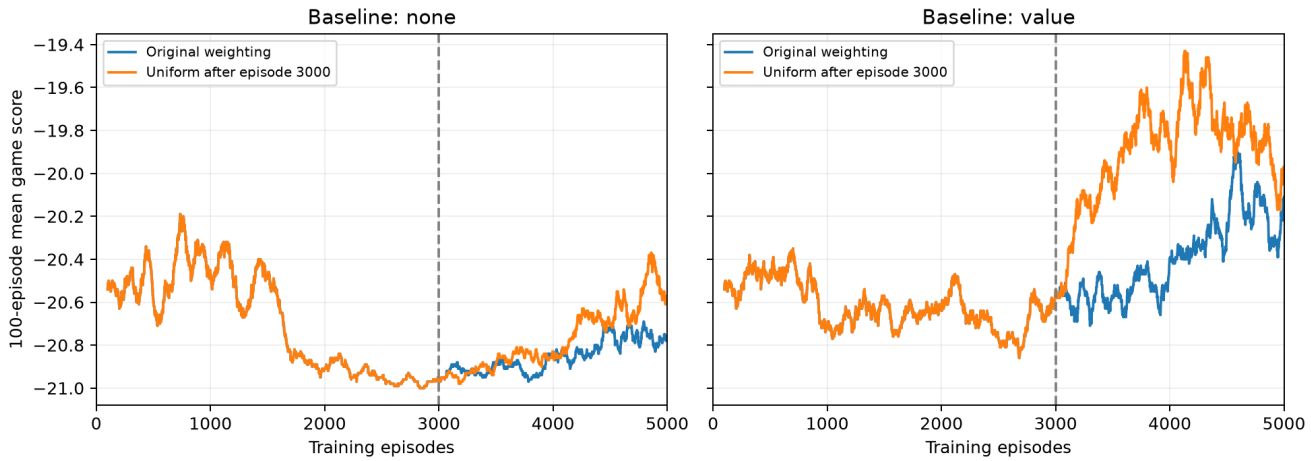


Figure 13. Original and uniform-weight training branches; the vertical line marks the shared 3,000-episode fork.

The changes are larger than rollout uncertainty for these particular policies, but the gains are small and the policies still play poorly. I learned that changing the weighting alone was not enough. These intervals describe evaluation variability, not variation between independently trained policies. I kept this practical weighting for later exploration and continued to report the strict-objective results separately.

11. Pong: motion representation at fixed budget

I next asked whether the network was struggling to infer motion from two binary frames. I represented appearing and disappearing pixels in separate binary channels; subtracting the channels recovers the signed frame difference. This keeps the CNN input shape unchanged and preserves motion direction through packed storage. Simply packing signed -1/0/1 differences would lose that information.

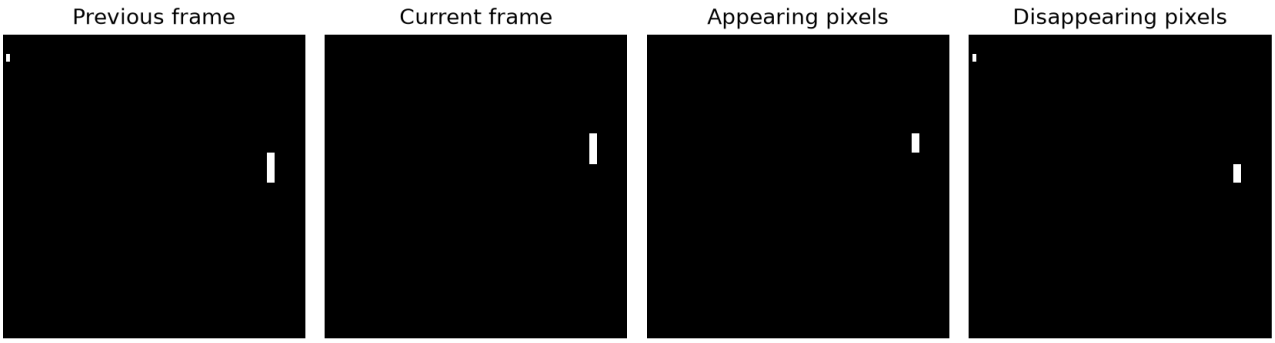


Figure 14. Real ALE observations and the two motion channels. Static pixels disappear from both motion channels.

A fair comparison starts both value-baseline models fresh: stacked frames versus split frame difference, actor lr=0.0001, critic lr=0.0003, uniform time weighting from episode 1, seed 0, 3,000 episodes and 500 evaluations. Reusing a stacked-frame checkpoint with the new encoding is rejected by the program.

Input	500-episode mean +/- SD
Stacked frames	-20.010 +/- 1.217
Split frame difference	-19.888 +/- 1.038

Motion minus stacked score = 0.122; approximate paired 95% interval [-0.018, 0.262]. The interval includes zero. This experiment does not establish a reliable advantage of motion input at this learning rate.

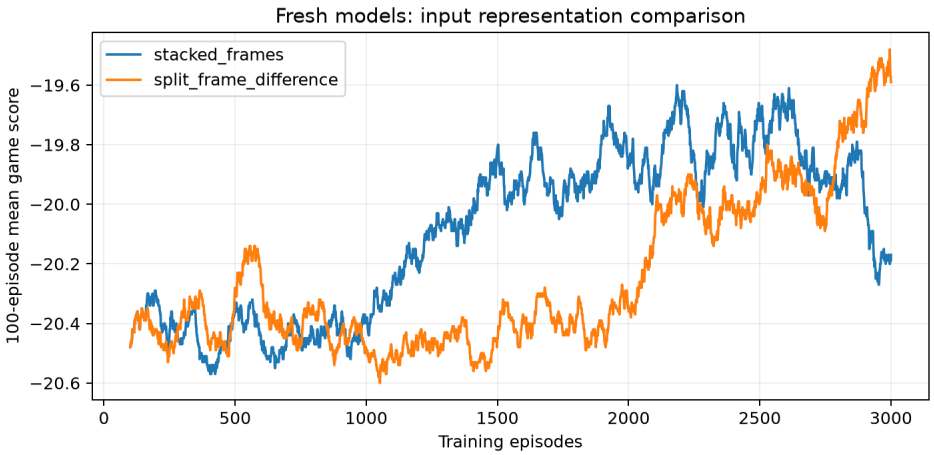


Figure 15. Fresh models under matched budgets. A visually higher training segment is not a substitute for the independent score comparison.

12. Pong: actor learning rate is the larger gain

The motion-input experiment did not produce a clear score improvement, so I returned to the actor learning rate. I held the motion representation, CNN, value baseline, uniform weighting, critic lr=0.0003, batch size 8, clipping at 5 and seed 0 fixed. I compared the completed fresh lr=0.0001 run with fresh runs at 0.0003 and 0.001. Each received 3,000 episodes, or 375 actor updates, and 500 evaluations.

Actor lr	Evaluation mean +/- SD	Gain over 0.0001
0.0001	-19.888 +/- 1.038	Reference
0.0003	-19.122 +/- 1.407	0.766
0.001	-7.742 +/- 6.311	12.146

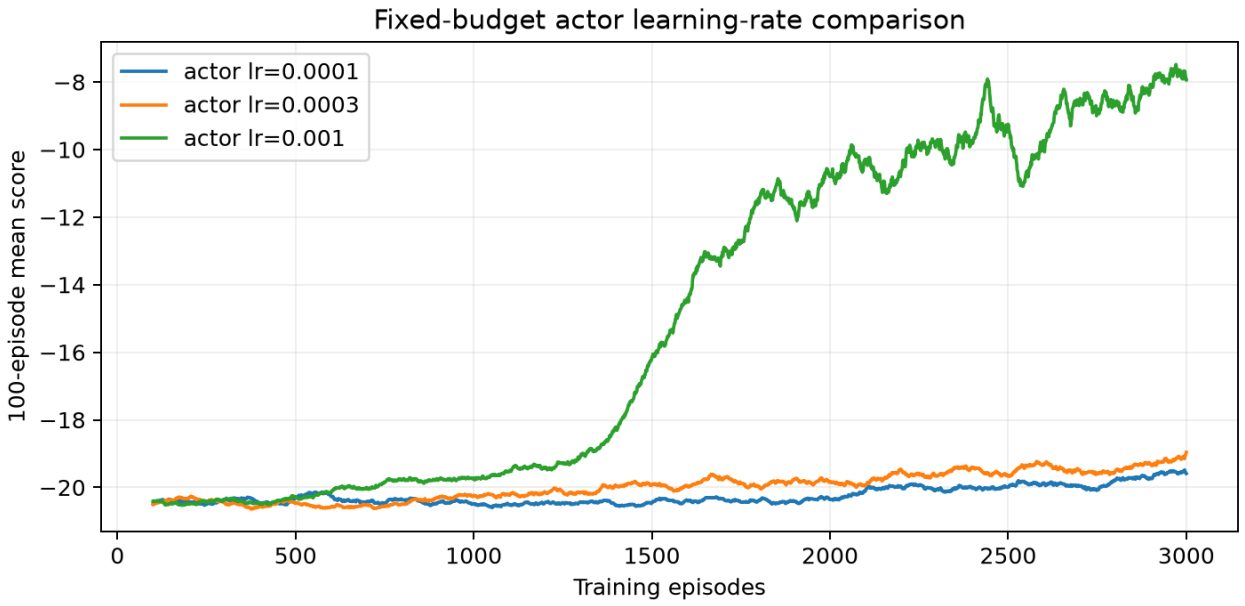


Figure 16. Matched training curves. The actor lr=0.001 configuration learns much faster within the fixed 3,000-episode budget.

The lr=0.001 gain is 12.146 score units, with approximate paired rollout interval [11.585, 12.707]. This is much larger than the earlier budget, weighting and representation gains. It is evidence for the tested actor learning rate within this specific practical configuration, not proof of its effect under the strict outer-discounted objective or every network.

The lr=0.001 result was encouraging, but I had used these evaluation scores to choose it. I therefore wanted independent training seeds before treating it as typical. Section 13 reports that check. I have not run a matched no-baseline version of this improved family, so I cannot attribute its improvement to the baseline alone.

12a. Improved Pong extension: selected seed 0

Configuration: ALE/Pong-v5, baseline=value, seed 0, gamma=0.99, actor Adam lr=0.001. Training: 3,000 episodes. Frozen stochastic-policy evaluation: 500 episodes.

Mean score	Sample SD	Random-policy mean
-7.742	6.311	-20.514

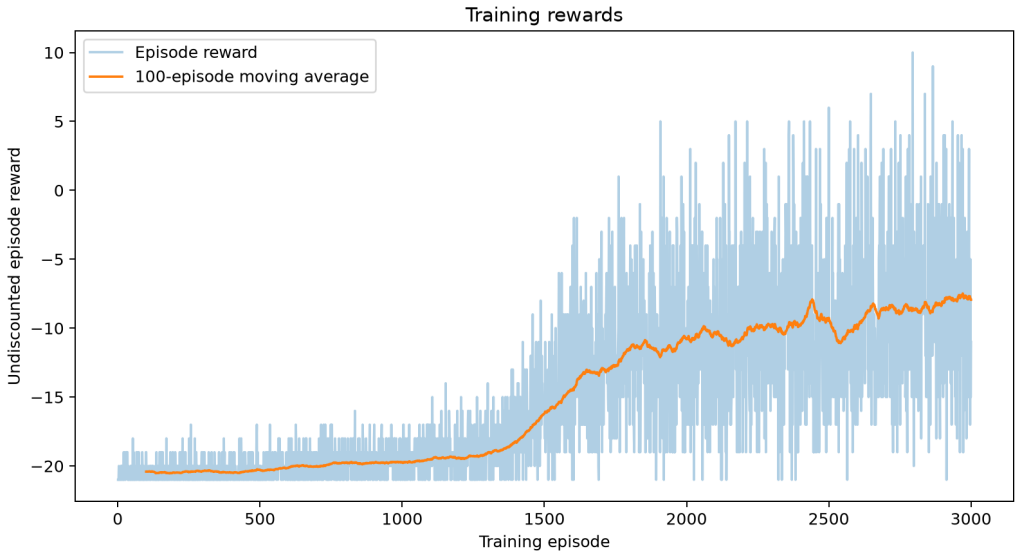


Figure 17. Training reward and trailing 100-episode mean. Source: results/pong_motion_lr001_value/training.csv.

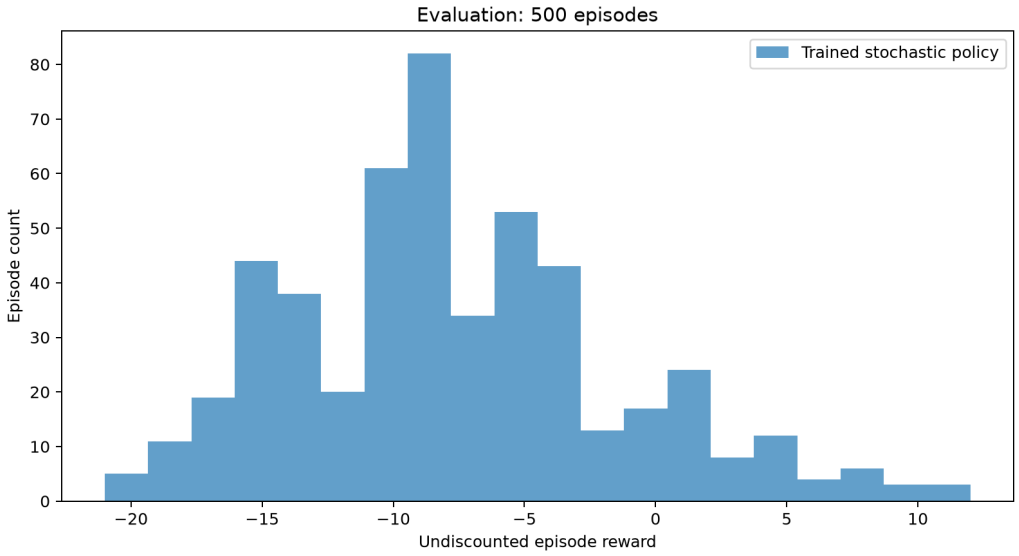


Figure 18. Histogram of the 500 frozen-policy game scores; random-policy reference is displayed separately. Standard deviations use ddof=1.

Supplementary uniform-time-weighted method, not the strict-objective core experiment. The selected policy scores -7.742 +/- 6.311; 60 of 500 evaluation scores are positive, and the maximum is +12. The mean remains negative, so the policy is not claimed to win reliably. Selection on these results makes independent seeds important.

13. Independent training-seed confirmation

After choosing lr=0.001 on seed 0, I kept the settings fixed and trained two new models at seeds 1 and 2. Each started with fresh networks and optimizers and used the same 3,000-episode budget. The seed changes initialization, action sampling and disjoint environment-seed ranges. Each model then received 500 evaluations and its own seed-matched random reference.

Training seed	Role	Mean +/- rollout SD	Positive scores / 500
0	Selection	-7.742 +/- 6.311	60
1	Confirmation	-9.250 +/- 3.668	9
2	Confirmation	-10.032 +/- 3.796	2

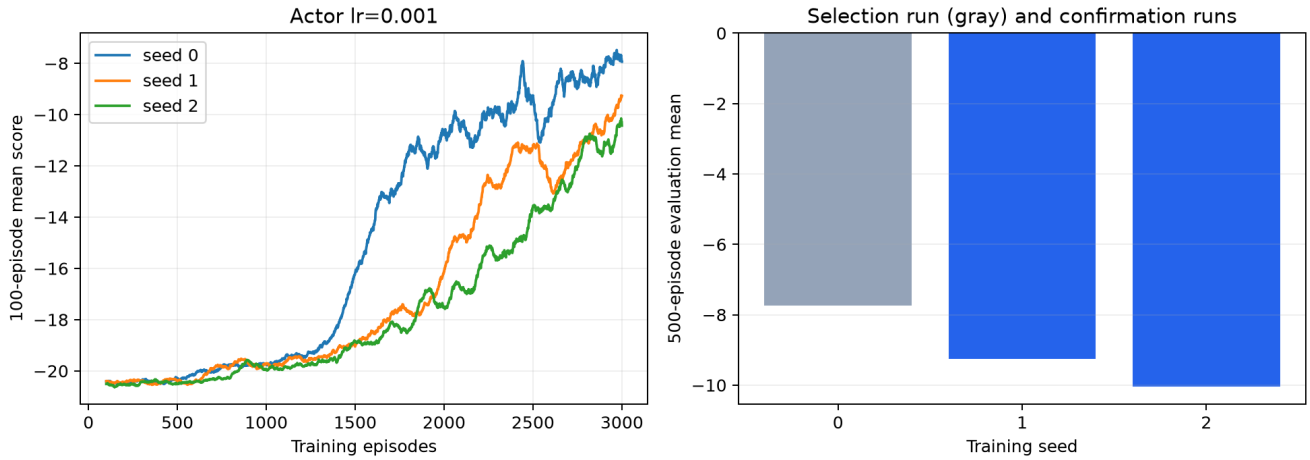


Figure 19. Individual training curves and independent 500-episode score means. Seed 0 was used for hyperparameter selection.

The independent seed-1/seed-2 mean is -9.641. The three-seed mean is -9.008; the sample SD between the three policy means is 1.164. These describe training-procedure variability separately from each policy's rollout SD.

Both confirmation runs improved substantially over their random references, which makes the improvement more convincing than the selected seed alone. Still, three training seeds are a small sample and all three means remain negative. I did not repeat the low-learning-rate control at seeds 1 and 2, so this is not a full learning-rate-by-seed study. Pooling the 1,500 evaluation scores would not turn them into 1,500 independently trained models.

Training seed ranges: 20,000,000 + 100,000*seed through that value +2,999. Evaluation ranges: 4,000,000 + 10,000*seed through that value +499. Training, evaluation and separate diagnostic ranges are disjoint for these reported runs.

14. Shuttle validation beyond basic unit tests

I did not want to accept the shuttle solution just because two functions returned the same policy: they could share a mistake. I checked predictions with known answers, compared different algorithms and used a simulator that applies the service and arrival events independently of the DP transition table. The table below records both what I expected and what actually happened.

Prediction / test	Observed result
Tiny problem: 5 states, 2 arrivals, horizon 4; recursive action/outcome tree vs DP	Maximum error 0.0
VI and PI must agree	Value max error 1.965e-09; 0 policy mismatches
PI must satisfy Bellman optimality	Residual 9.095e-13
T=500 approximates discounted infinite horizon	Value max error 2.979e-09
Free dispatch should serve positive queues	All positive states dispatch
Zero capacity should make paid dispatch useless	No state dispatches
gamma=0 should remove future effects	All values equal minimum immediate cost

Independent Monte Carlo cost validation

Initial queue	DP cost	Simulation mean	Standard error
0	576.670	576.769	0.156
12	672.892	673.024	0.153
13	676.670	676.848	0.157
15	676.670	676.286	0.157
200	3821.586	3821.739	0.375

For each initial queue, I compared the DP value with 40,000 independent 500-step simulations (seed 314159). Four of five comparisons fall inside their individual 95% intervals. State 15 is 2.44 standard errors below the DP value; I retained that discrepancy rather than hiding it. All five are inside simultaneous 95% Bonferroni intervals, and the omitted discounted tail is below 6.9e-8. This supports the coded transition and cost calculation within simulation error; it does not prove that my overflow assumption is the only reasonable model.

```
python src/shuttle.py --out runs/shuttle
python src/validate_shuttle_simulation.py
```

15. Part 1: does the policy gradient behave as predicted?

A small problem with an exact answer

A rising training curve would not tell me whether I had coded the right derivative. I used a two-step Bernoulli problem with p=sigmoid(theta), rewards a_0 and 2*a_1, gamma=0.95 and theta=0.3. Its expected objective is J=2.9p, so its derivative is 2.9p(1-p)=0.708929104. Enumerating all four action sequences gives the expected score-gradient estimate without Monte Carlo noise. With no baseline, the absolute error is 5.96e-8. A parameter in the actual CartPole actor also agrees with a central finite difference to 4.12e-12.

This checks the sign, reward-to-go and outer discount factor against a known mathematical answer. It is stronger evidence than checking tensor shapes, although it cannot establish that a CNN will learn Pong well.

CartPole: make behavior easier to predict

I reduced policy randomness using two hand-coded controllers on 20 reset seeds each. I expected a constant left push to fail quickly, while angle-and-angular-velocity feedback should keep the pole balanced. The mean rewards were 9.15 and 500.00 respectively. All feedback episodes ended at the time limit; constant-left episodes ended by physical failure. Reward equaled the number of steps. These observations check the state/action convention, reward scale and distinction between termination and truncation. They are diagnostic controllers, not learned-policy results.

Pong: check the action meaning in real frames

Prediction / experiment	Observed result
Action 2 should move the player paddle upward	Paddle center 34.5 -> 11.5
Action 3 should move it downward	Paddle center 34.5 -> 57.5
A simple ball-tracking controller should beat random	10-episode means: tracking -16.1, random -20.3

The tracking controller is still weak. Its result is useful because image-based feedback can affect the score in the expected direction, not because it proves the neural policy is correct. I also inspected real motion-channel images (section 11). Sign reconstruction and static-pixel cancellation have exact expected answers; they help detect an encoding mistake before blaming training budget.

For the learned policies, I used the matched budget, representation and learning-rate comparisons in sections 9-13. Some predictions failed: simply training longer and changing representation did not solve Pong. Keeping those outcomes is part of validating the algorithm on this task. Dense-versus-chunked gradient tests are supporting implementation checks, not replacements for these behavioral experiments.

```
python src/policy_gradient.py --test-only
python src/diagnose_cartpole_control.py
python src/diagnose_pong_control.py
python -m unittest discover -s src -p "test_pong_*.py"
```

15a. Looking at the first ten decisions

Aggregate scores can hide a reversed action or a broken observation. I inspected the first ten CartPole decisions from the feedback diagnostic at reset seed 9,500,000. The decision signal is u=angle+0.15*angular_velocity; action 1 is chosen when u is positive, otherwise action 0. The table uses the stored, measured states. Angles are radians and angular velocities are radians per second.

Step	Pole angle	Angular velocity	Signal u	Action	Reward
0	0.02105	0.03767	0.02670	1	1
1	0.02180	-0.24830	-0.01544	0	1
2	0.01684	0.05118	0.02451	1	1
3	0.01786	-0.23615	-0.01756	0	1
4	0.01314	0.06212	0.02245	1	1
5	0.01438	-0.22639	-0.01958	0	1
6	0.00985	0.07079	0.02047	1	1
7	0.01127	-0.21877	-0.02155	0	1
8	0.00689	0.07745	0.01851	1	1
9	0.00844	-0.21306	-0.02352	0	1

At step 0, the angle is positive and the angular velocity is also positive, so the controller pushes right. At step 1 the angle is still positive, but the angular velocity has become negative enough to reverse the decision signal. The action changes to left. This made it clear why looking only at pole angle would miss an important part of the state. All ten rewards are +1 and neither ending flag is set; the full episode later reaches the 500-step truncation.

What the early Pong trace can and cannot tell me

In the first ten tracking decisions at seed 8,000,000, rewards are all zero and the simple detector does not yet locate the ball. The controller falls back to a target at y=40. Its chosen actions are 3, 3, 2, 2, 3, 3, 3, 2, 2, 3. The recorded paddle centers begin 34.5, 36.5, 46.5 and 47.5. I do not expect every next position to follow the latest chosen action immediately: this Pong-v5 environment retains frame skipping and sticky actions. The fixed-action probes in section 15 are easier to interpret for movement direction.

This trace illustrates two task-specific issues: early zero rewards are normal in Pong, and action effects need not appear instantly. It does not demonstrate that the detector tracks the ball reliably or that the trained policy has learned. The independent game-score evaluations remain necessary.

The complete four-coordinate CartPole states and ending flags are saved in results/cartpole_control_diagnostic.json. The Pong observations, chosen actions and scoring events are in results/pong_control_diagnostic.json. Initial learned-policy traces are also retained as first_10_steps.json in the original experiment directories. These traces can be regenerated with the two diagnose commands in section 15.

15b. Part 2: does the baseline reduce variance?

What I expected, and how I tested it

My prediction was more specific than "the baseline should play better": subtracting an action-independent baseline should leave the expected gradient unchanged and may reduce its variance. In the exactly enumerable two-step problem, using the nonzero constant baseline 1.7 still matches the analytic derivative within 5.96e-8. This checks the cancellation argument separately from any critic-learning effect.

For the learned baseline, I froze the actor and critic and collected new trajectories on seeds starting at 8,000,000. On each identical trajectory, I computed the full actor gradient twice: once with reward-to-go alone and once after subtracting the frozen value prediction. Thus a difference cannot be explained by comparing two differently trained actors. I measured total gradient variance as the trace of the sample covariance across every actor parameter.

Frozen policy / samples	Plain variance	Baseline variance	Ratio (95% bootstrap interval)
cartpole, 64 trajectories	21499	33.83	0.00157 [0.00129, 0.00191]
pong, 16 trajectories	2326.8	281.96	0.12118 [0.07340, 0.29105]

The ratios are below one for both checkpoints, which supports variance reduction here. Intervals use 300 paired bootstrap resamples. CartPole uses the final 2,000-episode baseline checkpoint and 64 new trajectories; Pong uses the strict 1,000-episode checkpoint and 16. The Pong sample is small, so I treat its interval as exploratory. These results do not establish variance reduction for every checkpoint or for the later uniform-weight method.

Baseline-specific edge cases and what I learned

If the critic receives the actor gradient, it could change to reduce the actor loss instead of estimating returns. I detach its output and compute it before fitting the current data. Dense reference tests explicitly check that actor backpropagation leaves critic gradients unset. If a baseline is poorly calibrated, it need not reduce variance; I therefore measured the gradients directly. If the actor nearly always chooses one action, both variance estimates can be close to zero and a ratio becomes misleading. The old collapsed Pong checkpoint is flagged as numerically degenerate and is not used as convincing ratio evidence.

Gradient clipping is a separate stability intervention. The unbiased-baseline argument concerns the score-gradient estimator before clipping; it does not make every clipped optimizer step unbiased. Reference-gradient tests use a very large clipping threshold to compare the underlying gradients. My game-score tables remain essential: lower variance at the tested Pong checkpoint did not produce a strong Pong player. That was the most useful limitation of the baseline experiment.

```
python src/reproduce_validation.py
python src/diagnose_pong_activations.py
python src/audit_results.py
```

The reproduction command verifies the committed actor/critic capsules and repeats the analytic, finite-difference and paired frozen-policy experiments without retraining. Activation measurements additionally check that the later CNNs have nonzero ReLU activity; they are network-health evidence, not a substitute for score or variance measurements.

16. Edge cases, objective checks and limits

Part / edge case	Handling and evidence
Shuttle: s=0, empty dispatch	Never create negative passengers; dispatch still costs 100. Boundary test checks the immediate cost.
Shuttle: s=200 and overflow	Waiting yields 200 for every arrival; dispatch yields 186-190. Clipping and overflow assumption are explicit.
Shuttle: terminal time and ties	J_500=0; late-horizon actions can differ. Greedy ties choose wait consistently.
CartPole: termination or truncation	Both end the sampled episode; no value bootstrap is added. The finite 500-step return is evaluated.
Pong: restricted actions and resets	Only actions 2/3; two-frame history resets per episode. Split motion starts with zero change.
Sparse rewards / long episodes	Full-episode returns, no score-point reset. Exact gamma^t and uniform variants are explicitly distinguished.
Variable trajectory lengths / memory	256-step gradient chunks preserve the sum; Pong averages per episode, not by pooling all steps.
Non-finite or extreme gradients	Reject non-finite gradients; batched Pong clips norm at 5. Low-entropy/zero-variance failures are retained.
Warm-start continuation	Check episode count against CSV; preserve source prefix, optimizer and CPU sampler RNG; record source hashes.
Changing input representation	Reject cross-representation resume; use a fresh network and optimizer for fair comparisons.
Baseline fitting leakage	Detach baseline; compute before fitting the current data. Direct variance checks use frozen actor/critic.

Training objective and whole-game score can disagree

Strict-objective policy	Mean discounted return	Mean score
random	-1.4135	-20.415
none	-1.2554	-20.410
value	-1.2424	-20.740

I evaluated these strict-objective policies for 200 additional episodes each on seeds starting at 6,000,000. Their discounted returns improve over random even though their whole-game scores remain poor. This helped me understand why apparently valid gradient updates had not produced a good player. It does not tell me exactly which reward timings changed. Reproduce this check with python src/evaluate_pong_objective.py --episodes 200 --device cpu.

17. Reproduction, evidence and submission

Use Python 3.12. The measured game runs used PyTorch 2.14.1, Gymnasium 1.4.0, ALE 0.12.1, NumPy 2.5.4, Adam and one CPU thread. CPU was used for CartPole and historical runs, Apple MPS for batched Pong. The recorded lock file describes the actual training environment. Report rendering is separate from training; hardware and numeric differences can change stochastic trajectories.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-lock.txt
```

From the repository root, write repeats to fresh runs/ directories. The four commands below correspond exactly to the core report configurations. Every trained policy is evaluated for 500 episodes; --device cpu is available if MPS is unavailable.

```
python src/policy_gradient.py --game cartpole --baseline none --lr 0.001 --episodes 2000 --eval-episodes \
500 --out runs/cartpole_none
python src/policy_gradient.py --game cartpole --baseline value --lr 0.001 --episodes 2000 --eval-episodes \
500 --out runs/cartpole_value
python src/pong_batch.py --baseline none --lr 0.0001 --episodes 1000 --batch-episodes 8 --eval-episodes \
500 --device mps --out runs/pong_none
python src/pong_batch.py --baseline value --lr 0.0001 --episodes 1000 --batch-episodes 8 --eval-episodes \
500 --device mps --out runs/pong_value
```

The improved seed-0 extension can be repeated without changing source code. For independent confirmation change --seed to 1 or 2 and choose fresh output directories.

```
python src/pong_batch.py --baseline value --lr 0.001 --value-lr 0.0003 --episodes 3000 --eval-episodes \
500 --batch-episodes 8 --seed 0 --device mps --time-weighting uniform --representation \
split_frame_difference --out runs/pong_improved_seed0
```

The batch scripts run_learning_rate_comparison.py and run_seed_validation.py preserve completed results and reject duplicate manifests. They are one-time experiment controllers, not report-regeneration commands. For a repeat, use the explicit training command with fresh outputs. Exact historical source snapshots and SHA-256 manifests remain in results/. Atari ROMs and virtual environments are not committed.

```
python src/audit_results.py
python src/reproduce_validation.py
python src/prepare_report_figures.py
python src/build_report.py
```

No retraining is needed to regenerate this PDF or repeat the included actor/critic gradient-variance validation capsules. reports/report_config.json selects the core experiments. results/report_evidence.json ties every reported experiment to raw CSV and summary hashes. The report and required plots are in the public repository. Submit reports/assignment1_report.pdf to Canvas; updating GitHub does not itself submit Canvas.

18. What three additional months would add

Period	Predictions and experiments	Evidence to collect
Month 1: isolate causes	Run a no-baseline counterpart to the improved Pong family at matched seeds. Cross strict/uniform weights with actor lr; test constant vs learned baseline. Vary shuttle capacity, cost ratios, queue cap and overflow rule.	Paired seed effects, discounted and undiscounted returns; Bellman residuals; simulated costs and threshold changes.
Month 2: quantify robustness	Use at least 10 predeclared training seeds, separate tuning and confirmation seeds, and equal step/update budgets. Test observation noise and initial-state shifts. Study altered Atari sticky actions only as explicitly separate tasks.	Between-seed uncertainty and failure rates; held-out evaluation curves; robustness profiles without conflating environment changes.
Month 3: understand learning	Freeze early/mid/late checkpoints to measure baseline gradient variance. Compare architectures and motion encodings. Study nonstationary/heavy-tailed shuttle arrivals. Replicate selected models across CPU/MPS.	Full-gradient covariance traces and baseline calibration; sample and wall-clock efficiency; independent replication and model-assumption sensitivity.

With more time, I would begin with predictions before running experiments. For the shuttle, a higher dispatch cost should usually delay service, while higher holding cost should usually accelerate it; I would measure thresholds and investigate boundary exceptions. For the games, I would spend the first month on matched comparisons, then use more independent seeds and checkpoint-level variance measurements. This would help separate a repeatable improvement from a lucky configuration.

What I learned and what remains unresolved

The clearest lesson for me was that correctness and learning quality need different evidence. I have strong numerical support for the shuttle solution from a small recursive tree, Bellman checks and independent simulation. For policy gradients, I can verify the derivative and show lower variance without claiming that every trained player is good. In Pong, more episodes alone and the first representation change were insufficient; actor lr=0.001 in the practical motion-input family gave the largest tested improvement, and the confirmation seeds support that improvement.

I would still describe Pong as unfinished rather than solved. Its mean scores remain negative, the best configuration has only three training seeds, and it lacks a matched improved no-baseline control. Uniform actor weights also change the core objective estimator. I kept these limits explicit, preserved the failed runs and used measured results throughout. The next useful step would be to test the remaining explanations, rather than simply making the training run longer.

Sources and assistance

[R1] Course handout, mlds_assignment1.pdf, pages 1-4. Supplied assignment, due 26 October 2026 at 11:59 pm.

[R2] Farama Foundation. CartPole-v1 environment specification.
https://gymnasium.farama.org/environments/classic_control/cart_pole/

[R3] Farama Foundation. ALE Pong environment and action specification.
https://ale.farama.org/environments/pong/

[R4] OpenAI Spinning Up. Intro to Policy Optimization: reward-to-go and baselines.
https://spinningup.openai.com/en/latest/spinningup/rl_intro3.html

AI assistance was used for implementation, debugging and report preparation. All numerical experiment results are from actual executions and retained artifacts. The author should be able to explain the estimators, assumptions and validation evidence independently.

Appendix A. Complete measurement inventory

All formal training experiments below have 500 final-policy evaluation episodes. The explicitly named early diagnostic uses only 20 training and 2 evaluation episodes and is not an assignment-sized result. Other short smoke demos are kept separately. Sample SD is over evaluation episodes, not training seeds. Raw curves and histograms are in each result directory; 100-episode averages apply when enough training episodes exist.

Result directory (under results/)	Train	Eval	Mean	Sample SD
cartpole_none	2000	500	9.370	0.755
cartpole_none_lr0001	2000	500	300.240	97.968
cartpole_none_lr0001_seed1	2000	500	92.460	31.626
cartpole_none_lr0001_seed2	2000	500	106.554	28.962
cartpole_none_lr0003	2000	500	171.432	42.960
cartpole_value	2000	500	170.168	11.184
cartpole_value_lr0001	2000	500	220.218	45.618
cartpole_value_lr0001_seed1	2000	500	362.986	121.367
cartpole_value_lr0001_seed2	2000	500	214.266	69.583
pong_batch_none	1000	500	-20.390	0.827
pong_batch_value	1000	500	-20.720	0.564
pong_budget3000_none	3000	500	-20.952	0.214
pong_budget3000_value	3000	500	-20.548	0.693
pong_budget5000_none	5000	500	-20.724	0.563
pong_budget5000_value	5000	500	-20.170	0.916
pong_motion_lr0003_value	3000	500	-19.122	1.407
pong_motion_lr001_value	3000	500	-7.742	6.311
pong_motion_lr001_value_seed1	3000	500	-9.250	3.668
pong_motion_lr001_value_seed2	3000	500	-10.032	3.796
pong_none	200	500	-21.000	0.000
pong_repr_motion_value	3000	500	-19.888	1.038
pong_repr_stack_value	3000	500	-20.010	1.217
pong_uniform5000_none	5000	500	-20.444	0.798
pong_uniform5000_value	5000	500	-19.954	1.141
pong_value	200	500	-21.000	0.000
pong_value_early_diagnostic	20	2	-20.500	0.707

Requirement map: shuttle MDP/assumptions and three requested figures - sections 1-2; Part 1 and Part 2 algorithms/baseline - section 3; all four required reward curves and 500-episode histograms - sections 4-7; failure-driven improvements - sections 8-13; quantitative validation - sections 14-15b; per-part edge cases - section 16; reproducibility/GitHub/Canvas - section 17; three additional months - section 18.